PARKING User Manual

This is the user manual for how to set-up, maintain, and run the PARKING application. As an overview, running the system requires maintaining a database and establishing connections with physical cameras to obtain data.

Setting up a camera:

Prerequisites:

- An 8MP IMX219 camera
- A “Raspberry 4 Pi” module
- Any continuous power source
    - The camera and Raspberry Pi must always be powered on
- Materials to fix the system in place

Steps:

1. Construct a system that contains the camera, raspberry pi, and power source all together.
2. Find a high, suitable place to station the system.
3. Determine the optimal angle to place the system for the best coverage of your parking lot.
4. Affix the system to the location so that it will be entirely stationary and facing a fixed angle at all times.
5. You’re all set! Now the camera needs to be added into the database for it to be useful.

Setting up the database:

The database needs to be up and running at all times, so it should be set up on a server environment with constant uptime.

The database is written in Python using version 3.13.5, and must have access to the **ultralytics** and **json** packages. Download the files from the “Processing” folder and place them into the database on the server.

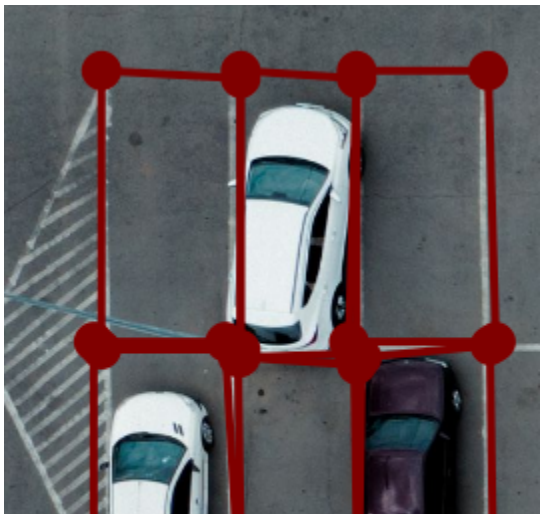
To start, establish a timer on the system with some timer interrupt. Every time a number of seconds equal to the timer interrupt has passed, the database will signal every camera to take a new picture and process it to update their data. Then, a flag will be raised to notify any active apps to pull this new data from the server.

Adding to the database:
Any additions to the database have to be made manually.

The system for adding to the database is currently under development. This section will be updated once changes are finalized.

Annotating images:
There currently no effective system for accurately detecting and marking parking spots, so it has to be done manually.

A reference image must be taken from a camera, annotated for parking spots, and then passed into the database. Since the reference image is based on a fixed angle and simply contains pixel coordinates of parking spots, it’s imperative that the camera never moves or else the reference will be useless. If the camera is ever moved, a new reference must be established.



Shown above is an example of proper image annotation. On the left is the original image, and on the right is the annotated version. Each spot must be recorded using a polygon with exactly four vertices.

```
{
  “shapes”: [
      “points”: [
          [vertex1_x, vertex1_y],
          [vertex2_x, vertex2_y],
          [vertex3_x, vertex3_y],
          [vertex4_x, vertex4_y]
      ]
  ]
}
```

Above is the hierarchy displayed by a properly formatted JSON file outputted after the annotation of a reference image.

As shown, there is a list “shapes” that contains a list “points”. Inside the “points” list is any number of pairs of pixel coordinates (vertex_x and vertex_y are doubles). If the image was annotated correctly, there should be exactly four pairs.

Additional data may be contained in the JSON file, but it won’t be read by the program.

To annotate our images, we use an image annotation software called LabelMe. However, any annotation method works so long as a .JSON file following the guidelines is properly produced.